

Project Proposal Form, 2025
Institute of Information Technology (IIT)
University of Dhaka

Student's Name:	RATUL SIKDER		
Student's Roll:	2506102	Phone:	01790532862
Project Title: Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery Motivation: Modern software engineering teams spend an estimated 40–60% of their time on maintenance, debugging, and manual incident resolution rather than building new features. Automated pipelines — CI/CD workflows, data integration jobs, and microservice deployments — break silently and frequently. A schema changes upstream. A dependency shifts. An API contract is violated. By the time engineers discover the failure, downstream damage is already done: corrupted data, broken dashboards, stalled releases, and financial loss. Existing AI-assisted tools such as GitHub Copilot, automated test generators, and deployment monitors address individual engineering tasks in isolation. None replicate the collaborative structure of a real engineering team and none close the loop between detecting a failure and autonomously repairing it. The engineer remains the bottleneck — pulled into every incident, writing every fix, and approving every deployment manually. Nexis addresses both gaps simultaneously. It models the software engineering team as a system of role-specialised AI agents and embeds a self-healing intelligence that autonomously detects, diagnoses, repairs, and validates failures — with most decisions made automatically or reported directly to the admin, reducing human involvement to a single one-click approval step only where critical. Project Description: 1. User Management and Access Control (Intelligent) Nexis provides a complete user-facing authentication and role management system. Users can register, log in, and manage their profiles securely. The system supports multiple roles — Admin, Engineer, and Viewer — each with distinct access			

permissions and dashboards. Role assignment is handled intelligently: the system analyses user behaviour patterns and access history to recommend role adjustments to the Admin automatically. Login anomalies (unusual location, time, or device) are flagged by the intelligent monitoring layer without manual intervention. Admins can approve, revoke, or reassign roles from a centralised control panel with full audit visibility.

2. Admin Dashboard and Intelligent System Control

The Admin Dashboard serves as the central command interface for the entire Nexis platform. Admins can monitor all active agents, running pipelines, ongoing incidents, and pending approvals in real time. The dashboard surfaces intelligent alerts — anomalies, risk scores, and recommended actions — generated automatically by the Self-Healing Loop. Most routine decisions (low-risk patch approvals, non-critical rollbacks, health threshold adjustments) are resolved autonomously by the system and reported to the Admin as completed actions rather than requiring manual input. High-risk decisions are escalated with full context for one-click approval.

3. Dual-Layer Multi-Agent Architecture

Nexis operates across two fully implemented coordinated layers connected by a shared memory bus and typed inter-agent messaging protocol. **Layer 1 — Execution Team** consists of five role-specialised agents: the Architect Agent (system design and API contracts), Backend Agent (code generation and repair), QA Agent (test generation and validation), DevOps Agent (deployment and health monitoring), and Data Engineer Agent (data pipelines and schema management). **Layer 2 — Self-Healing Loop** consists of four monitoring agents: Sentinel (fault detection), Pathfinder (causal diagnosis), Synthesiser (repair generation), and Validator (shadow execution and approval queue). All nine agents are fully built and deployed as part of this project. Every step in both layers is instrumented, logged, and reported to the Admin dashboard automatically.

4. Architect Agent — System Design and Contract Enforcement

The Architect Agent decomposes high-level requirements into a structured system blueprint. It defines API schemas, data contracts, and service boundaries that all other agents must comply with. It acts as the system-wide source of truth — automatically detecting and blocking contract violations proposed by other agents before they propagate downstream. All violations are reported to the Admin with a

plain-language explanation and recommended resolution, without requiring manual investigation.

5. Backend Agent — Intelligent Code Generation and Repair

The Backend Agent implements features against Architect-defined contracts using LLM-guided code synthesis. It applies the minimal diff principle — generating the smallest valid code change that satisfies the requirement. When the Self-Healing Loop identifies a code failure, repair tasks are automatically delegated to the Backend Agent by the Synthesiser. Low-risk patches are applied and deployed autonomously after shadow validation; high-risk changes are surfaced to the engineer for one-click approval with full context.

6. QA Agent — Automated Property-Based Testing

The QA Agent auto-generates unit, integration, and property-based tests from API contracts and historical behavioural data, covering edge cases that human-written tests typically miss. It runs continuously — not just on demand — and feeds structured defect reports back to the Backend Agent automatically. Test failures are classified by severity: critical issues trigger immediate Self-Healing Loop activation; non-critical issues are batched and reported to the Admin in a daily intelligent summary.

7. DevOps Agent — Deployment Health and Autonomous Rollback

The DevOps Agent manages CI/CD pipeline execution and deployment sequencing. It continuously monitors post-deployment signals — latency, error rates, resource utilisation, and service uptime. When health thresholds are breached, it intelligently decides whether to trigger an automatic rollback or apply a targeted patch, based on failure severity and available fix confidence scores. All decisions are logged and reported to the Admin dashboard in real time, with only high-stakes deployments requiring explicit human approval.

8. Data Engineer Agent — Schema Management and Drift Recovery

The Data Engineer Agent builds and maintains ETL/ELT pipelines aligned with Architect-defined data contracts. It tracks schema versions continuously and propagates contract changes across dependent pipelines. Schema drift — the most frequent production failure class — is detected automatically, a repair patch is synthesised and shadow-validated, and the fix is either applied autonomously (for low-risk drift) or queued for Admin approval (for breaking changes). All schema change events are logged with full lineage traceability.

9. Sentinel Agent — Proactive Fault Detection

The Sentinel Agent ingests telemetry from the DevOps and Data Engineer Agents continuously. It applies statistical process control over Apache Flink streaming to identify the exact moment a failure occurs — within seconds, not hours. It classifies faults by type and severity, emits a structured fault report, and immediately activates the Pathfinder Agent. Admins receive a real-time notification for every detected fault, with severity level and affected component, without needing to manually monitor logs.

10. Pathfinder Agent — Causal Fault Localisation

The Pathfinder Agent traverses the system dependency graph using counterfactual causal inference (do-calculus) to identify root cause versus symptoms automatically. It produces a ranked hypothesis list with confidence scores — for example: "Schema field user_id renamed in source system — 94% confidence." The diagnosis is delivered to the Admin dashboard and passed to the Synthesiser Agent for repair generation, replacing 2–6 hours of manual log investigation with a sub-minute automated diagnosis.

11. Synthesiser Agent — Autonomous Repair Generation

The Synthesiser Agent generates 2–3 candidate repair patches via LLM-guided inductive program synthesis, constrained by formal system contracts and the failure evidence. It automatically routes each patch to the correct Layer 1 agent for execution — schema fixes to the Data Engineer Agent, code patches to the Backend Agent. Each patch includes a plain-language explanation of the fix. Simple, low-risk repairs are applied autonomously after shadow validation and reported as resolved to the Admin. Complex or high-risk repairs are escalated for approval.

12. Validator Agent — Shadow Pipeline Execution

The Validator Agent deploys each candidate patch into a Docker-isolated shadow pipeline running in real-time parallel with production. It executes property-based test suites, compares shadow output against historical data invariants, and automatically selects the highest-confidence patch. Rejected patches are discarded with a log entry. Approved patches are either autonomously deployed (low-risk) or presented to the engineer as a one-click approval with the shadow test results, diff, and plain-English summary — ensuring no unvalidated fix ever reaches production.

13. Intelligent Reporting and Notification System

Nexis generates automated reports summarising all agent activity, incidents detected, repairs applied, patches approved, and system health trends. Admins receive a real-time notification feed and a daily intelligent summary with actionable insights. Engineers receive targeted alerts only for incidents requiring their decision — eliminating alert fatigue from low-value notifications. Reports are available in the Admin Dashboard and can be exported. All reports are generated automatically by the system — no manual compilation is required.

Strategy & Timeline: The development strategy for Nexis follows an iterative, modular approach across four phases. Phase 1 covers user management, admin dashboard, and system architecture. Phase 2 builds the core self-healing loop, Backend Agent, and Data Engineer Agent. Phase 3 completes the remaining execution agents — Architect, QA, and DevOps — and implements the RLHF fine-tuning loop from engineer feedback. Phase 4 covers intelligent reporting, full evaluation, and final submission. All nine agents across both layers are fully built and deployed as part of this project. Every module is built with intelligent automation at its core — decisions are resolved autonomously where safe, and reported to the Admin where human judgement is required.

The project Gantt chart is given below:

Week	Task No / Module	Description	Dependencies
Phase 1 — User System, Admin Interface & Architecture			
1	1. Project Setup & Architecture Design	System architecture design, agent communication protocol, development environment setup	None
2	2. User Registration & Login	Implement secure user registration, login, and JWT-based authentication	Task 1
3	2. User Registration & Login	Finalise login system, password reset, and session management	Task 1
4	3. Role Management & Access Control	Implement Admin, Engineer, and Viewer roles with intelligent role recommendation to Admin	Task 2
5	4. Admin Dashboard	Build Admin control panel — agent status, incident feed, pending approvals, system health	Task 3
6	4. Admin Dashboard	Integrate intelligent alerts and autonomous decision reporting into Admin dashboard	Task 4
Phase 2 — Self-Healing Loop & Execution Agents			
7	5. Shared Memory & Agent Messaging	Implement Chroma vector store, relational state layer, and Pydantic typed inter-agent contracts	Task 1
8	6. Sentinel Agent	Build streaming fault detection using Apache Flink and statistical process control	Task 5
9	6. Sentinel Agent	Fault classification, severity scoring, and real-time Admin notification integration	Task 6
10	7. Pathfinder Agent	Build Neo4j dependency graph with OpenLineage automated lineage tracking	Task 6
11	7. Pathfinder Agent	Implement DoWhy causal inference for root cause ranking and confidence scoring	Task 7
12	8. Synthesiser Agent	LLM-guided patch synthesis with formal contract constraints and agent delegation logic	Task 7
13	9. Validator Agent	Docker shadow pipeline execution engine and Hypothesis property-based testing integration	Task 8
14	9. Validator Agent	Autonomous low-risk patch deployment and high-risk approval queue for Admin	Task 9
15	10. Backend Agent	LLM code generation, minimal diff repair, and Synthesiser delegation integration	Tasks 8, 9

16	11. Data Engineer Agent	ETL pipeline construction, schema drift detection, and autonomous repair with Admin escalation	Tasks 8, 9
Phase 3 — Remaining Agents, RLHF & Full Integration			
17	12. Architect Agent	System blueprint generation, API contract definition, violation detection and blocking	Tasks 6–11
18	13. QA Agent	Property-based test generation from contracts, defect reporting, and continuous test loop	Task 12
19	14. DevOps Agent	CI/CD orchestration, runtime health monitoring, intelligent rollback decision engine	Tasks 12, 13
20	15. RLHF Fine-Tuning Loop	Engineer approval feedback pipeline — model fine-tuning from approval, rejection, and modification history	Tasks 9, 12–14
21	16. Self-Healing Loop Full Integration	End-to-end integration of all 9 agents with Admin reporting, notification, and audit trail	All tasks
Phase 4 — Intelligence, Evaluation & Submission			
22	17. Intelligent Reporting & Notifications	Automated report generation, real-time notification feed, and daily Admin intelligent summary	Task 16
23	18. Human Approval Interface	One-click engineer approval UI with diff view, shadow results, and plain-English explanation	Task 16
24	19. Evaluation & Testing	Fault injection testing (schema drift, API violations, dependency breakage), baseline comparisons	All tasks
25	19. Evaluation & Testing	NASA-TLX user study, MTTR and patch correctness measurement, performance tuning	Task 19
26	20. Final Integration, Report & Submission	End-to-end system testing, final project report, presentation slides, and project submission	All tasks

Languages or Tools to be used:

- Agent Orchestration: LangGraph / AutoGen
- Backend Language: Python
- Inter-Agent Messaging: Pydantic typed schemas
- Shared Memory: Chroma (vector store) + PostgreSQL (relational store)
- Fault Detection (Sentinel): Apache Flink + Statistical Process Control
- Dependency Graph (Pathfinder): Neo4j + OpenLineage
- Causal Inference (Pathfinder): DoWhy / EconML
- Program Synthesis (Synthesiser): Claude API / GPT-4 with constrained prompting
- Shadow Execution (Validator): Docker-isolated pipeline replay engine
- Property-Based Testing: Hypothesis (Python) / fast-check (JavaScript)
- CI/CD (DevOps Agent): GitHub Actions + ArgoCD
- Infrastructure: Terraform
- Frontend / Admin Dashboard: React.js
- IDE / Editor: Visual Studio Code

Supervisor's Name: _____

Signature of the supervisor: _____

Date: _____

Before the Midterm Presentation 1:

I confirm that the progress is satisfactory and I am forwarding it for the midterm presentation.

Signature of the supervisor: _____

Date: _____

Proposal Presentation Feedback (if any) :

Midterm 1 Presentation Feedback (if any) :

Midterm 2 Presentation Feedback (if any) :